NPUMeter: Automatic Operator Optimization for Ascend NPU with Accurate Analytical Performance Models

WEICHUANG ZHANG, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
YUFEI SHANGGUAN, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
HAIBO XU, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
YUTING MAI, Huawei Technologies Co Ltd, Shenzhen, China
QIULIANG WANG, Huawei Technologies Co Ltd, Shenzhen, China
CHEN CHEN, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
YI LI, Huawei Technologies Co Ltd, Shenzhen, China
QUAN CHEN, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
WENCHAO DING, College of Intelligent Robotics and Advanced Manufacturing, Fudan University, Shanghai, China
JIERU ZHAO, School of Computer Science, Shanghai Jiao Tong University, Shanghai, China
MINYI GUO, College of Computer Science and Technology, Guizhou University, Guiyang, China and School of Computer Science, Shanghai Jiao Tong University, Shanghai, China

With the rapid development of AI and deep learning, computational demands are increasing significantly. While GPUs excel in parallel computing, they fall short in terms of energy efficiency, specialization, and processing latency. In contrast, Neural Processing Units (NPUs), such as the Ascend NPUs, designed specifically for deep learning tasks, demonstrate superior performance. However, the architecture specialization makes operator development more challenging, leading to a reliance on manual tuning and optimization, which incurs significant time cost and developing effort. To address this issue, we propose NPUMeter, an automatic operator optimization framework for Ascend NPUs built upon accurate and comprehensive analytical performance models. NPUMeter comprises two components: (1) an analytical performance model that accurately estimates operator latency on NPU given different configurations of optimization parameters; (2) an efficient design space exploration (DSE) algorithm that automatically searches for the optimal parameter configuration in a

Weichuang Zhang and Yufei Shangguan contributed equally to this research.

This work was supported by the National Natural Science Foundation of China (NSFC) under Grant 62472273 and Grant 62232015.

Authors' Contact Information: Weichuang Zhang, Yufei Shangguan, and Haibo Xu, School of Computer Science, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: 1064080006@sjtu.edu.cn, sgyufeiii@sjtu.edu.cn, xuhaibo23@sjtu.edu.cn; Yuting Mai, Huawei Technologies Co Ltd, Shenzhen, Guangdong, China; e-mail: maiyuting@huawei.com; Qiuliang Wang, Huawei Technologies Co Ltd, Shenzhen, Guangdong, China; e-mail: wangqiuliang@huawei.com; Chen Chen, School of Computer Science, Shanghai Jiao Tong University, Shanghai, Guangdong, China; e-mail: chen-chen@sjtu.edu.cn; Yi Li, Huawei Technologies Co Ltd, Shenzhen, Guangdong, China; e-mail: alick.li@huawei.com; Quan Chen, School of Computer Science, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: chen-quan@cs.sjtu.edu.cn; Wenchao Ding, College of Intelligent Robotics and Advanced Manufacturing, Fudan University, Shanghai, Shanghai, China; e-mail: dingwenchao@fudan.edu.cn; Jieru Zhao (corresponding author), School of Computer Science, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: zhao-jieru@sjtu.edu.cn; Minyi Guo, College of Computer Science and Technology, Guizhou University, Guiyang, Guizhou, China and School of Computer Science, Shanghai Jiao Tong University, Shanghai, Shanghai, China; e-mail: guo-my@cs.sjtu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).
ACM 1544-3973/2026/06-ART78
https://doi.org/10.1145/3820380

large design space within minutes. Experimental results demonstrate that NPUMeter achieves high estimation accuracy, with an average error below 5%. It effectively generates near-optimal configurations for various operators, achieving up to a 1.46× performance speedup compared to the configuration generated by the Ascend C compiler while reducing the DSE time from hours to minutes.

CCS Concepts: • **Computing methodologies** → **Modeling methodologies**

Additional Key Words and Phrases: NPU, performance model, analytical method, design space exploration

ACM Reference Format:
Weichuang Zhang, Yufei Shangguan, Haibo Xu, Yuting Mai, Qiuliang Wang, Chen Chen, Yi Li, Quan Chen, Wenchao Ding, Jieru Zhao, and Minyi Guo. 2026. NPUMeter: Automatic Operator Optimization for Ascend NPU with Accurate Analytical Performance Models. *ACM Trans. Arch. Code Optim.* 23, 2, Article 78 (June 2026), 26 pages. https://doi.org/10.1145/3820380

1 Introduction

With the rapid advancement of **artificial intelligence** (**AI**), the demand for computing systems that deliver both high performance and energy efficiency has grown substantially. While **Graphics Processing Units** (**GPUs**) have long served as the primary hardware for AI acceleration due to their massive parallelism [1, 12–15], the escalating complexity and scale of modern AI workloads expose their limitations in energy efficiency, architectural specialization, and processing latency. In contrast, **Neural Processing Units** (**NPUs**) [3] have emerged as domain-specific accelerators explicitly optimized for deep learning computation. By tightly coupling hardware microarchitecture with the computational patterns of neural networks, NPUs achieve significantly higher performance per watt and deliver better overall execution efficiency. Representative architectures such as Google TPU [2], Cerebras WSE [9], Graphcore IPU [8], Microsoft Brainwave [11], and Huawei Ascend [10] demonstrate the rapid evolution of NPU architectures, underscoring their increasing importance in large-scale AI deployment.

Despite their architectural advantages, NPUs pose significant challenges for operator implementation and optimization. The intricate organization of compute units, together with the multi-level on-chip and off-chip memory hierarchy, creates a large design space, especially for emerging architectures such as Ascend NPU [10]. Figure 1 illustrates the design space of a matrix multiplication operator on the Ascend NPU, where each point corresponds to a distinct design configuration. Developers must meticulously tune numerous parameters, such as tiling dimensions and sizes, allocated double buffer sizes, and iterative computation order, to achieve high data locality and minimize computational latency. However, identifying an optimal configuration on NPUs typically requires labor-intensive and iterative *onboard profiling*, which remains time-consuming and error-prone even for experienced experts. As shown in Table 1, exhaustive design space search on NPUs is computationally prohibitive, since each input shape of a given operator requires several hours of onboard profiling to evaluate all candidate configurations. In practice, the diversity of operators and tensor shapes in emerging applications further increases the complexity of the optimization process. To reduce development effort, it is essential to employ a more efficient DSE methodology for rapid exploration and to develop accurate performance models that enable fast and reliable evaluation on NPUs.

Existing methods. Prior research has investigated performance modeling techniques for NPUs. PREMA [4] and AI-MT [5] focus on systolic-array–based NPUs and construct analytical models that predict the latency of individual DNN layers. Layer-Puzzle [6] further employs mathematical modeling to estimate end-to-end latency on multi-core systolic arrays based on specific parallelization strategies and tiling configurations. Although the regular computational structure of systolic

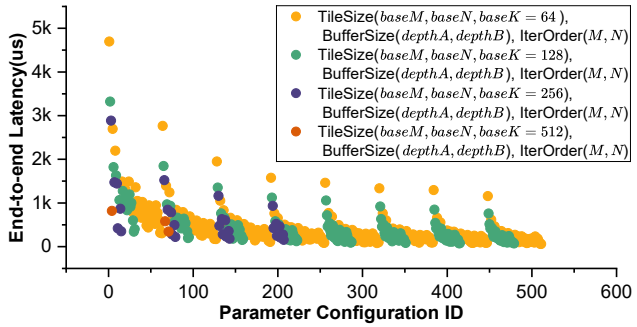


Fig. 1. Exhaustive design space search for matrix multiplication.

Table 1. Overhead of Exhaustive Design Space Search on NPU, Based on Onboard Performance Profiling

Operator	Input shape	Design Space	Exhaustive Search Time (hours)
GEMM	(512, 512, 1024)	2^9	2.28
GeGLU	(512, 512, 1024)	2^{10}	2.85
FFN	(512, 512, 512)	2^{10}	3.56

arrays makes performance modeling relatively straightforward, such models cannot be directly applied to more complex Ascend NPU architectures. Verrocchio [33] builds a performance model for Huawei's DaVinci AI cores, whose data paths and memory hierarchies are much more complex. It estimates the latency of computation and memory operations by dividing the corresponding data volumes by the measured bandwidth or computational throughput of each hardware unit. While this unified modeling scheme simplifies implementation, it ignores the distinct behaviors of compute and memory units, leading to significant prediction errors for various operators. Beyond analytical approaches, WMDRS [7] adopts a learning-based strategy that collects runtime workload statistics such as AI core utilization, operating frequency, and memory bandwidth together with device metadata to train a model for predicting the execution time of operators. However, the collection of these runtime features and the associated model-training process introduce nontrivial overhead. Moreover, the accuracy of such learning-based models is highly sensitive to feature selection, making them unreliable across diverse operators and input shapes.

Recent studies have also adopted DSE techniques [16] [17] [28] [29] [30] [31] [32] to improve design efficiency and quality on NPUs. Timeloop [16] provides a framework for mapping DNN workloads onto spatial architectures, employing heuristic-based exploration to search mapping configurations. However, it does not consider coordinated optimization across different compute units, nor the impact of computation ordering on overall performance, which can result in suboptimal designs. Similarly, GAMMA [17] performs operator-level mapping on a fixed spatial architecture via genetic algorithms, but its search is confined to a narrow hardware space and lacks holistic consideration of computation and data movement, making it difficult to extend to more complex Ascend NPU architectures.

Main challenges. Emerging NPU architectures incorporate diverse compute units and complex multi-level memory hierarchies, such as Ascend NPU [10], significantly increasing architectural complexity. As a result, existing analytical performance models and DSE strategies designed for spatial architectures cannot be directly applied. Some studies have explored learning-based modeling, but their black-box nature inherently restricts fine-grained performance analysis and

often suffers from limited accuracy. Compounding the issue, the exponentially expanded design space significantly hinders systematic design automation and optimization.

Our approach. To address all these challenges, we propose NPUMeter, an automated operator optimization framework that combines an analytical performance model with efficient design space exploration. Specifically, the performance model analyzes the execution process of compute units and the data interaction between various levels of the memory hierarchy, enabling precise performance modeling for different operators. Based on accurate and fast evaluation from the performance model, our DSE algorithm explores the design space rapidly and finds the high-performance parameter configuration efficiently.

Our main contributions are summarized as follows:

- We propose NPUMeter, an optimization framework that automatically tunes operator performance and significantly reduces design effort.
- We develop the first analytical performance model for Ascend NPU, capable of precisely estimating both computation and memory latency across diverse operators.
- Leveraging the fast and accurate estimation from our analytical model, we construct the design space and introduce an efficient DSE algorithm to automatically search for optimal parameter configurations.
- Experimental results show that NPUMeter achieves high estimation accuracy, with errors remaining below 5% across diverse operators. The proposed DSE algorithm reduces exploration time from hours to minutes while delivering up to a 1.46× speedup compared to the configuration generated by the Ascend C compiler.

2 Background and Motivation

2.1 Architecture of Ascend NPU

We validate our method on the Huawei Ascend 910 NPU, which features a Da Vinci architecture [10] as illustrated in Figure 2. Each AI core incorporates a *Cube Core* dedicated to matrix operations, a *Vector Core* for vector computations, and a *Scalar Unit* responsible for scalar tasks such as address calculation and loop control. The AI Cores adhere to a **Single Program Multiple Data** (**SPMD**) parallel paradigm, where multiple cores execute the same instruction sequence (indicated by orange arrows). Blue arrows depict the dataflow within the multi-level memory hierarchy. *L0A* and *L0B* function as input buffers for the Cube Core, while *L0C* serves as its output buffer. The *L1 Cache* and *Unified Buffer* constitute the second level of memory, fetching data from off-chip memory (HBM or *L2 Cache*) via the **memory transfer engine** (**MTE**).

2.2 Dataflow on Ascend NPU

The dataflow of operators varies depending on the type of computation core employed, as shown in Figure 2. This leads to different execution patterns.

Cube Core Dataflow. The cube core dataflow consists of four stages. First, the MTE2 stage transfers input matrix tiles from the HBM or L2 cache to the L1 cache. Second, the MTE1 stage reads matrix tiles of input matrices A and B from the L1 cache into the L0A and L0B caches, respectively. Third, the cube core computation stage reads operands from L0A/L0B, performs calculations, and stores results in the L0C cache. Finally, during the Fixpipe write-back stage, result tiles are transferred from the L0C cache back to off-chip memory.

Vector Core Dataflow. The vector core dataflow consists of three stages: MTE2, vector core computation, and write-back. The MTE2 stage is similar to that of Cube operators, transferring data from off-chip memory (HBM or L2 cache) to the **Unified Buffer** (**UB**). The vector core then executes vector operations, followed by writing results back from UB to HBM.

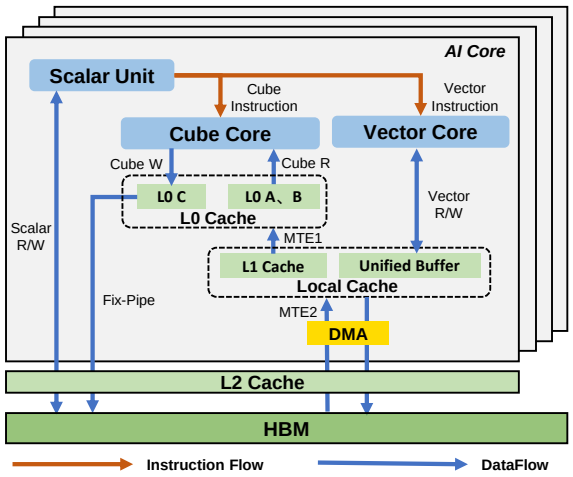


Fig. 2. Ascend NPU based on Da Vinci architecture.

Cube-Vector (CV) Fusion Dataflow. Cube-Vector (CV) fusion dataflow refers to those that require collaboration between cube core and vector core, corresponding to operators that involve both matrix and vector computations. Data synchronization between the two cores is facilitated by allocating a workspace in the L2 cache, where data interactions occur.

2.3 Format Conversion in Cube Core

We first clarify the concept of tiles and blocks in Ascend NPU processing, and then introduce the format conversions required for cube core execution.

Tile and Block. The cube core processes one matrix *tile* each time, as shown in Figure 3(d). Each tile is further divided into multiple *blocks*, which are processed by the cube core in a pipelined manner. Format conversions are required to maintain correctness during cube core execution.

Data Format Definitions. There are four kinds of data formats used during cube core execution: *ND*, *Nz*, *Zz*, and *Zn*, as shown in Figure 3(d). *ND* refers to the standard row-major layout commonly used in HBM. *Nz* is a block-based layout where elements inside each block follow a row-major order, while the blocks themselves are arranged in column-major order. *Zz* denotes a layout where both the elements within each block and the blocks themselves follow a row-major order. *Zn* denotes a layout where both the elements within each block and the blocks themselves follow a column-major order.

ND-to-Nz in MTE2. During the MTE2 stage, the matrix tiles undergo a conversion from *ND* format to *Nz* format. This transformation changes the storage order of the matrix elements from a row-major order to a column-major order among blocks, while maintaining row-major storage within each block.

Nz-to-Zz/Zn in MTE1. During the MTE1 phase, the matrix tiles of input A and B undergo distinct format conversions: *Nz-to-Zz* and *Nz-to-Zn* conversions, respectively. The *Nz-to-Zz* conversion transforms the inter-block storage order from a column-major to a row-major arrangement. Conversely, the *Nz-to-Zn* transformation alters the intra-block storage order from row-major to column-major organization.

2.4 Motivating Example

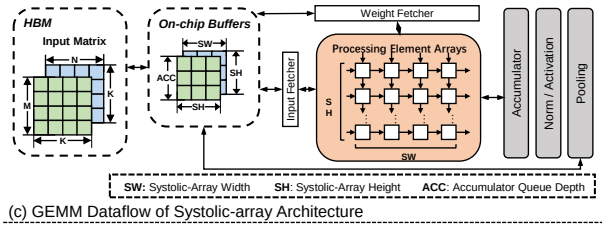
We use matrix multiplication as a motivating example to highlight the unique challenges in performance modeling and design space exploration for emerging NPUs. Traditional spatial architectures typically employ a regular 2D array of **processing elements (PEs)**, as exemplified by the systolic

```
void matmul(
    float A[M][K], float B[K][N],
    float C[M][N]
){
    bufferA = L1.allocate(depthA1);
    bufferB = L1.allocate(depthB1);
    while (matmul.Iter(Order::FirstM))
    {
      //tiling tensor A,B
      tileA(baseM, baseK), tileB(baseK, baseN)
        = Tiling(A, B);
      //Copy data from HBM to L1 cache
      bufferA.Copy_to_L1_Nd2Nz(tileA);
      bufferB.Copy_to_L1_Nd2Nz(tileB);
      //Copy data from L1 cache to L0A/B cache
      Copy_to_L0A_Nz2Zz(tileA);
      Copy_to_L0B_Nz2Zn(tileB);
      //Cube core computation
      tileC = matmul(tileA, tileB);
      // write back
      Copy_to_HBM(tileC);
    }
}
```

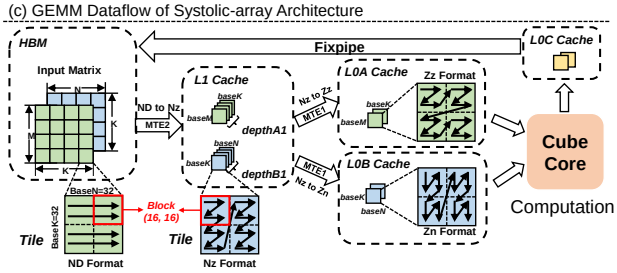
(a) Pseudo Code of GEMM

Parameters	Configuration
Tensor Tiling Size	baseM, baseN, baseK
Double Buffer Size in L1 Cache	depthA1, depthB1
Iteration Order	FirstM, FirstN

(b) Parameter Configurations of Operator



(c) GEMM Dataflow of Systolic-array Architecture



(d) GEMM Dataflow of Ascend Da Vinci Architecture

Architecture	GEMM memory access time	GEMM computation time
Systolic-Array	$\frac{SH * (ACC + SW)}{BW}$	$(SW + SH + ACC - 1)$
Ascend Da Vinci	$\frac{baseK * (baseM + baseN)}{BW} + L(ND2Nz + Nz2ZzNZ/Zz)$	$Func(baseM, baseK, baseN)$

(e) Comparison of Different NPU Performance Modeling

Fig. 3. Motivating example. (a) shows the code snippet of matrix multiplication described in Ascend C. (b) presents corresponding tunable parameters. (c) and (d) show the architectural features and dataflow of systolic array-based NPU and Ascend NPU, respectively. (e) presents performance modeling differences by formulating computation and data transfer time.

arrays in Google TPUs (Figure 3(c)). In such architectures, performance can be straightforwardly estimated as a function of PE array dimensions (*SW, SH*), accumulator queue depth (*ACC*), memory bandwidth (*BW*), and data volume, as approximated by the formulas in Figure 3(e).

Challenges in Performance Modeling. In contrast to traditional spatial architectures, emerging NPUs like Ascend exhibit more complex execution patterns, which utilize different computation cores instead of a PE array and incorporate deeper memory hierarchies. Figure 3(d) illustrates the execution dataflow of the *Cube Core* on Ascend NPU, revealing three primary challenges in performance modeling. *(1) A complex multi-level memory hierarchy.* Data must traverse several stages, starting from off-chip memory (HBM or the L2 cache), moving into the L1 cache, and then into the dedicated L0A and L0B input caches via memory transfer engines. It is then processed by the cube core, stored in the L0C output cache, and ultimately written back to HBM via the Fixpipe stage. Accurately modeling this process requires detailed consideration of each transfer stage and the intermediate buffer management between them. *(2) Mandatory data format conversions.* As shown in Figure 3(d) and the accompanying pseudo-code in Figure 3(a), Ascend NPU mandates specific data layouts for computations in the cube core, triggering format conversions during MTE transfers. These conversions incur non-negligible latency overhead and must be explicitly captured in the data transfer model, as reflected in the analytical expression in Figure 3(e). *(3) Tile-wise and block-wise pipelined computation.* The cube core processes one matrix tile each time, with each tile further divided into blocks that are processed in a pipelined manner. Additionally, the execution of multiple tiles can also be pipelined if data arrives timely. The block size is typically fixed at 16×16, while tiling sizes are tunable parameters that significantly impact execution performance. Consequently, an accurate pipeline model is essential to capture both tile-wise and block-wise computation behaviors.

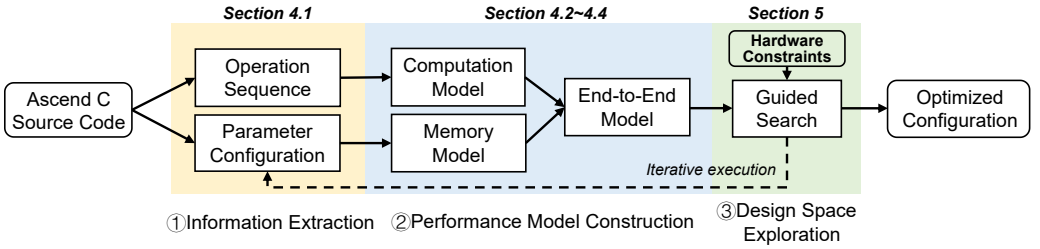


Fig. 4. Framework Overview.

Design Space Analysis. These architectural complexities also directly impact design space exploration. As highlighted in Figure 3(b), developers can tune several key parameters for GEMM on Ascend NPUs. Unlike systolic arrays, where the tile sizes often align with fixed PE row/column sizes, Ascend NPUs enable flexible configuration of tile sizes across multiple matrix dimensions (specified by parameters like *baseM*, *baseN*, and *baseK* in Figure 3(a)). Furthermore, double buffer sizes within the L1 cache (e.g., *depthA1* and *depthB1* for input matrices) must be carefully allocated to optimize data reuse and minimize costly data movement, guided by the chosen memory access order. The combination of these tunable design points forms a large design space. Critically, the complex interactions between different types of parameters, such as tiling strategy and buffer allocation, exacerbate the challenge of finding the optimal configuration for peak performance. NPUMeter utilizes a guided search strategy to efficiently explore the large design space defined by key parameters, enabling rapid identification of near-optimal configurations.

3 Framework Overview

Figure 4 illustrates the execution flow of NPUMeter. NPUMeter accepts Ascend C source code as input and generates an optimized parameter configuration tailored to maximize performance. First, the information extraction phase parses the source code to identify the operation sequence and tunable parameter configurations. Leveraging this data, it constructs distinct computation and memory models, which are subsequently integrated into a unified end-to-end model to predict overall execution latency. Finally, the design space exploration phase utilizes a guided search algorithm to determine the optimal configuration, strictly adhering to hardware constraints (e.g., L0 cache capacity) to guarantee design validity. The DSE algorithm iteratively refines the parameter combinations until it converges on a high-performance configuration. Detailed discussions of these components are presented in the corresponding sections.

4 Performance Model in NPUMeter

In this section, we present the details of our performance model, including the computation model, the memory model, and the end-to-end model.

4.1 Inputs of Model

Given the input source code, NPUMeter first extracts two key components for performance modeling. The first is the operator execution sequence, derived by analyzing operator types and their data dependencies to construct the computation flow. For the cube core, this sequence corresponds to a chain of matrix-multiplication operations, while for the vector core, it determines the structure of the computation pipeline such as the example in Figure 6(c). The second component is the parameter configuration, which specifies tiling and double buffer sizes and determines pipeline behavior, such as the total number of tiles and the potential stalls. Together, these components serve as inputs to the performance models.

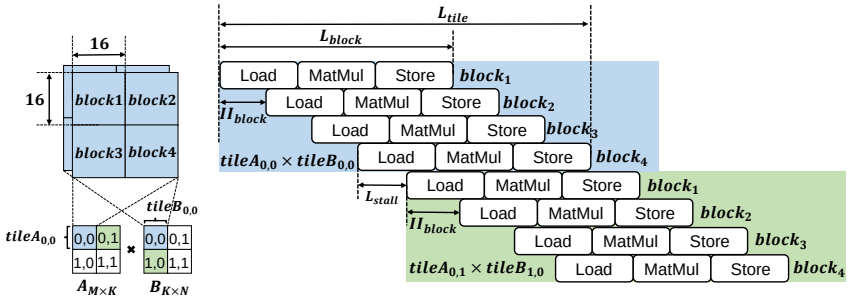


Fig. 5. Computation pipeline of the cube core.

4.2 Computation Model

The computation model estimates the execution latency of operators. Leveraging the extracted source code information, NPUMeter builds distinct computation models for the cube core and vector core, as these units handle the majority of computation tasks.

Cube Core Computation Model. The cube core fetches matrix tiles from the L0A and L0B caches, performs matrix computations, and writes the results back to the L0C cache. As illustrated in Figure 5, the cube core performs matrix computation tile by tile, and each tile is further partitioned into multiple *blocks*. A *block* corresponds to the maximum amount of data the cube core can process per instruction issue, i.e., the core's native computation granularity. Blocks within a tile are executed in a deeply pipelined manner. Therefore, the latency of computing a single matrix tile can be formulated as

$$L_{tile} = L_{block} + II_{block} \times \left(\frac{baseM \times baseN \times baseK}{16 \times 16 \times 16} - 1 \right), \tag{1}$$

where L_{tile} and L_{block} denote the clock cycles required to compute one tile and one block, respectively. II_{block} is the initiation interval of the block-level pipeline, which is typically 1 cycle. The parameters *baseM*, *baseN*, and *baseK* represent the tiling sizes of each matrix tile along the corresponding matrix dimensions. Since the side length of a block is fixed at 16 in Ascend NPU (Figure 5), the trip count of the pipeline model, i.e., the number of blocks for each tile computation, can be calculated by dividing $16 \times 16 \times 16$. When the tile dimensions are not divisible by 16, padding is required to extend the dimensions to the nearest multiple of 16, which may introduce a small amount of additional latency and memory overhead.

When processing consecutive tiles, computation on the next tile can only begin once its required input data is available in the L0A/B Cache. If this data is not ready (i.e., it still needs to be fetched from external memory like L2 cache or HBM), the cube core computation pipeline will stall until the data arrives, as illustrated in Figure 5. This potential stall between tiles must be considered when modeling the total latency. Therefore, the computation latency for consecutive tiles can be formulated as

$$\begin{aligned} L_{cube} &= L_{tile} + II_{tile} \times (Num_{tile} - 1) \\ &= L_{allblocks} + L_{allstalls} \\ &= L_{block} + II_{block} \times \left(\frac{M \times N \times K}{16 \times 16 \times 16} - 1 \right) \\ &\quad + L_{stall} \times (Num_{HBMtrans} - 1), \end{aligned} \tag{2}$$

where $L_{allblocks}$ represents the computation latency of all the blocks for this matrix computation without any stall, and $L_{allstalls}$ represents the latency of all the potential stalls between tiles, as

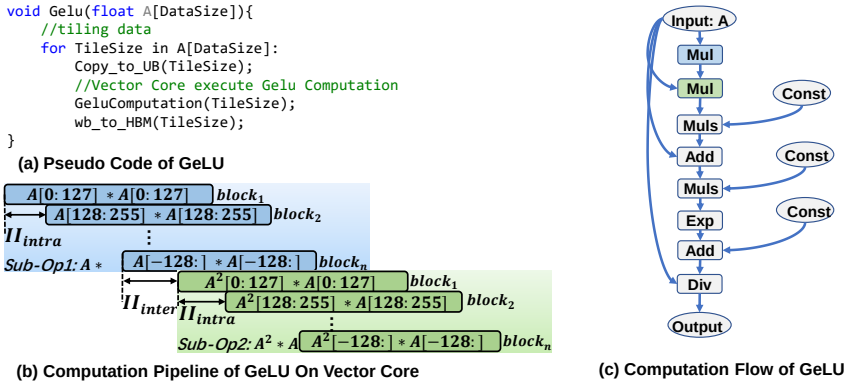


Fig. 6. The execution pipeline for one tile on the vector core. GeLU is selected as the example.

illustrated in Figure 5. Parameters M, N, and K denote the shape of the input matrices. L_{stall} represents the stall latency between consecutive tiles, which is determined through profiling experiments. $Num_{HBMtrans}$ denotes the total number of transfers of all input tiles from HBM to on-chip memory, which is formulated as

$$Num_{HBMtrans} = \frac{(M+N)\times K}{(baseM+baseN)\times baseK} \tag{3}$$

The number of stall events correlates with $Num_{HBMtrans}$ because data transfers from HBM to L1 are typically the performance bottleneck due to limited bandwidth. Consequently, the cube computation for each tile must wait until its input data is available, resulting in a stall. In contrast, transfers from L2 to L1 and from L1 to the cube input buffers (L0A/L0B) are fast enough to overlap with computation and do not introduce stalls in the cube pipeline. It is worth noting that this pipelining-based modeling approach is not restricted to the Ascend architecture. It can also be applied to other NPUs or accelerators that employ dataflow or pipelined execution. In such cases, architecture-specific parameters such as L_{block}, II_{block}, and L_{stall} can be obtained through profiling or microbenchmarking to instantiate the model.

Vector Core Computation Model. The vector core handles vector computations by decomposing them into sequences of basic sub-operators, such as *Add*, *Mul*, and *Div*. For example, the GeLU [18] operator depicted in Figure 6(c) comprises 13 sub-operators to complete. When processing one tile of input, the vector core executes sub-operators in order, with each sub-operator processed in a pipelined manner, as shown in Figure 6(b). For each sub-operator, data blocks with a fixed size of 256 bytes are processed, and the initiation interval between consecutive blocks is denoted as II_{intra}. Then the latency of each sub-operator, L_{subop}, is formulated as

$$L_{subop} = L_{block} + II_{intra} * \left(\frac{TileSize}{256} - 1\right), \tag{4}$$

where L_{block} represents the latency to complete the computation for the first block. Notably, we observe that II_{intra} exhibits variation across different sub-operator types. For instance, the II_{intra} for an FP16 multiplication operation (*Mul*) is two clock cycles, whereas an FP16 addition (*Add*) completes in just one clock cycle. These specific values are obtained through empirical profiling.

When processing consecutive sub-operators, the vector core requires a switching latency (II_{inter}) before initiating the next sub-operator computation, as shown in Figure 6(b). Therefore, the latency

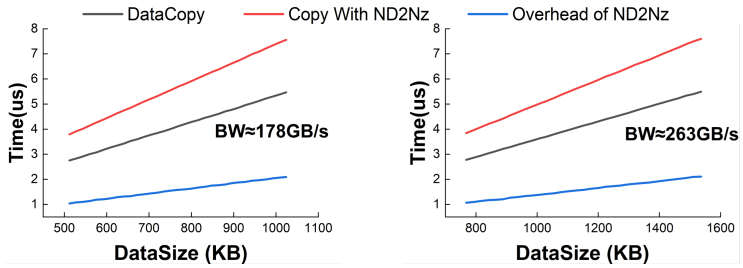


Fig. 7. Bandwidth profiling of MTE2. To investigate the impact of tile size on MTE2 bandwidth performance, we selected two representative cases from our experimental results: the left-side figure illustrates bandwidth behavior with 8KB tile size, while the right-side employs 12KB tile.

of one vector operation for one tile can be modeled as

$$\begin{aligned} L_{vec} &= L_{allblocks} + L_{inter} \\ &= L_{block} + \sum_{1}^{n} \left(II_{intra_i} * \left(\frac{TileSize}{256} - 1 \right) \right) + \sum_{1}^{n-1} II_{inter_i} \end{aligned} \tag{5}$$

where L_{block} corresponds to the computation latency required for the first block, $\sum_{1}^{n}(II_{intra_i} * (\frac{TileSize}{256} - 1))$ is the sum of the initiation intervals between the instructions within sub-operators, and $\sum_{1}^{n-1} II_{inter_i}$ is the sum of the inter-operation initiation intervals.

Similar to the computation pipeline of the cube core shown in Figure 5, the vector core also processes tiles in a pipelined manner. Therefore, considering multiple tiles, the total computation time required by the vector core to process the entire input can be modeled as

$$L_{allVec} = L_{block} + \left(\sum_{1}^{n} \left(II_{intra_i} * \left(\frac{TileSize}{256} - 1 \right) \right) + \sum_{1}^{n-1} II_{inter_i} \right) \times Num_{tile} \tag{6}$$

where Num_{tile} denotes the total number of input tiles.

4.3 Memory Model

The memory model estimates memory access time across memory hierarchies. Given a parameter configuration, NPUMeter calculates the data volume for each data transfer stage and constructs memory models for MTE2, MTE1, and Fixpipe, respectively.

MTE2 Memory Model. MTE2 refers to the data transfer between the HBM/L2 cache and the L1 cache. Due to the necessity of format conversion during data transmission, as introduced in Section 2.3, the memory access time of MTE2 cannot be modeled simply as a function of $\frac{DataSize}{Bandwidth}$. Therefore, we conducted experiments to reveal the impacts of format conversion, including both raw data transfer and format-converted transfer, as shown in Figure 7. The experimental results reveal several key observations:

(1) Format Conversion Overhead. Format conversion introduces a quantifiable overhead that reduces effective bandwidth. For a constant matrix shape (M, N), the conversion overhead demonstrates a linear relationship with data volume, characterized by a shape-dependent scaling coefficient.

(2) Bandwidth Stability. When transferring matrices of a fixed shape, both raw and format-converted bandwidth remain nearly constant regardless of the total data volume.

Consequently, the memory access latency of MTE2 is formulated as follows:

$$T_{MTE2} = T_{DataCopy} + T_{ND \to Nz}, \tag{7}$$

where $T_{DataCopy}$ is the time required for data transfer and $T_{ND \to Nz}$ denotes the format conversion overhead. It is important to note that there is an L2 cache between the HBM and the L1 cache. Data is fetched from off-chip memory to the L1 cache only on the first access and is subsequently cached in the L2 cache for reuse in later computations. Therefore, the data transfer time is modeled as

$$T_{DataCopy} = \frac{DataSize_{HBM}}{BW_{HBM}} + \frac{DataSize_{L2}}{BW_{L2}}, \tag{8}$$

where $DataSize_{HBM}$ and $DataSize_{L2}$ denote the sizes of data that are stored in the corresponding memory hierarchy, and BW_{HBM} and BW_{L2} represent the corresponding achieved bandwidths obtained via profiling.

$T_{ND \to Nz}$ exhibits a linear relationship with the data transfer volume when processing matrices of fixed dimensions. This relationship can be formally expressed as

$$T_{ND \to Nz} = \alpha \times DataSize \tag{9}$$

where α is the format conversion overhead coefficient obtained through profiling. *Datasize* denotes the volume of data transferred from off-chip to on-chip memory.

MTE1 Memory Model. The MTE1 stage reads matrix tiles of input matrices A and B from the L1 cache into the L0A and L0B caches, respectively. Similar to MTE2, the memory model of MTE1 can be expressed as follows:

$$T_{MTE1} = \frac{DataSize}{BW_{L1}} + T_{Nz \to Zn/Zz} \tag{10}$$

where $T_{Nz \to Zn/Zz}$ is the delay caused by format conversion.

Fixpipe Memory Model. During the Fixpipe write-back stage, result tiles are transferred from the L0C cache back to off-chip memory. Similar to MTE2, the memory model of Fixpipe can be expressed as follows:

$$T_{Fixpipe} = \frac{DataSize}{BW_{L0C}} + T_{Nz \to ND} \tag{11}$$

where $T_{Nz \to ND}$ is the delay caused by format conversion.

4.4 End-to-end Model

Finally, an end-to-end model is constructed to estimate the overall latency of operators based on three kinds of dataflow on Ascend NPU, as introduced in Section 2.2.

Cube Core Dataflow. For the cube core dataflow, an ideal pipeline is shown in Figure 8(a). The memory access, computation, and write-back stages are perfectly overlapped, allowing the pipeline to operate at its minimum $II = T_{MTE2}$. The corresponding ideal pipeline latency is then expressed as

$$\begin{aligned} T_{Ideal_pipeline} &= Depth + II \times (Num_{tile} - 1) \\ &= T_{MTE2_{tile}} + T_{MTE1_{tile}} + T_{Cube_{tile}} + T_{Fixp_{tile}} + II \times (Num_{tile} - 1), \end{aligned} \tag{12}$$

where $Depth$ denotes the pipeline depth, including the time for the four stages of a tile, which can be computed with our computation and memory models, and II denotes the inter-tile initiation interval.

However, this perfect overlap can only be achieved when data is transferred between L2 cache and L1 cache for the MTE2 stage. When data is transferred between HBM and L1 cache, the lower bandwidth of HBM compared to L2 cache often leads to stalls in the cube core pipeline, as illustrated



(a) Ideal Pipeline (b) Actual Pipeline

Fig. 8. Pipeline of cube core dataflow.

in Figure 8(b). Similarly, the MTE1 stage can also introduce stalls during execution in practice. Therefore, the end-to-end latency of a cube core dataflow is formulated as

$$T_{CubeOnly} = T_{ideal_pipeline} + T_{MTE2stall} + T_{MTE1stall}, \tag{13}$$

where $T_{ideal_pipeline}$ denotes the execution time of the ideal pipeline, $T_{MTE2stall}$ denotes the stall latency caused by HBM access delays, which is obtained by computing the difference between the execution latency of MTE2 and MTE1. Similarly, $T_{MTE1stall}$ denotes the stall latency caused by memory accesses to the L1 cache, which is obtained by computing the difference between the execution latency of MTE1 and the cube core computation.

Vector Core Dataflow. Using double buffer or ping-pong buffer, the vector core dataflow can be perfectly pipelined, and the computation time constitutes its end-to-end latency, overlapping most of the reads and writes. Thus, the latency of the vector core dataflow is formulated as

$$T_{VecOnly} = T_{MTE2firstRead} + T_{MTE2lastWrite} + T_{comp}, \tag{14}$$

where $T_{MTE2firstRead}$ refers to the HBM memory access time for the first block, $T_{MTE2lastWrite}$ denotes the write-back time for the last block, and T_{comp} represents the total computation time for all blocks processed by the vector core. The values of $T_{MTE2firstRead}$ and $T_{MTE2lastWrite}$ are estimated using the memory model of MTE2 Equation (7) as introduced in Section 4.3.

Cube-Vector Fusion Dataflow. CV fusion dataflow refers to operators that require collaborative execution between the cube core and the vector core, typically involving both matrix and vector computations. Figure 10(a) illustrates an example using the GeGLU operator [19], where the cube core first performs matrix multiplication, and the vector core subsequently applies GeLU and element-wise multiplication on these intermediate results. The total latency of a CV fusion operator is determined by the relative per-tile processing speeds of the two cores. Figure 9 shows the two possible dataflow patterns. When the cube core produces intermediate results no slower than the vector core consumes them ($T_{ctile} \leq T_{vtile}$), the dataflow follows the pattern in Figure 9(a). In this case, the second cube core computation that writes to double buffer 1 must wait for the first vector core computation reading from the same buffer to finish, causing stalls on the cube side. The resulting latency equals the time for the cube core to complete the first tile (T_{ctile}) plus the total standalone execution time of the vector core ($T_{VecOnly}$). Similarly, when the cube core produces intermediate results faster than the vector core consumes them ($T_{ctile} > T_{vtile}$), the overall latency equals the total standalone execution time of the cube core ($T_{CubeOnly}$) plus the time for the vector core to process the last tile (T_{vtile}). Thus, the latency of CV fusion dataflow can be expressed as

$$T_{CVFusion} = \begin{cases} T_{ctile} + T_{VecOnly} & (T_{ctile} < T_{vtile}) \\ T_{CubeOnly} + T_{vtile} & (T_{ctile} > T_{vtile}) \end{cases}, \tag{15}$$

where $T_{CubeOnly}$ and $T_{VecOnly}$ denote the standalone execution time of the cube core and the vector core, and T_{ctile} and T_{vtile} denote the per-tile processing time of each core, respectively.

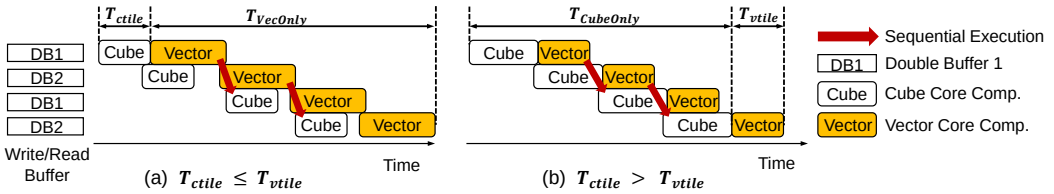


Fig. 9. Two patterns of CV fusion dataflow.

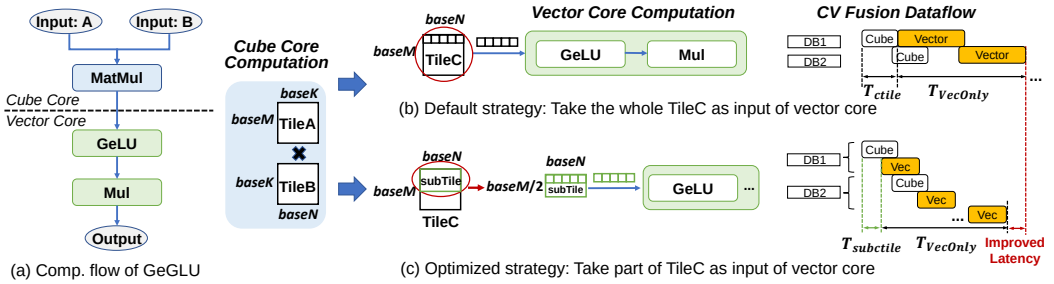


Fig. 10. Optimization of CV fusion dataflow.

We observe that this dataflow can be further optimized. As illustrated in Figure 10(b), the GeLU operator follows the $T_{ctile} \leq T_{vtile}$ dataflow pattern. In the default execution strategy, the vector core waits until the cube core finishes producing the entire `TileC` before starting its own computation, resulting in no overlap between the two cores. This leads to underutilization of available pipeline parallelism. However, in fact, the vector core can start execution as soon as the cube core generates a partial result. For example, in Figure 10(c), once the cube core finishes producing a `subTile`, the vector core can begin processing it. This enables overlap between cube core and vector core computation. Under this finer-grained tiling strategy, the total latency of this CV fusion operator becomes $T_{subctile} + T_{VecOnly}$, which effectively reduces the overall execution latency and improves performance.

4.5 Profiling-Derived Parameters

In this section, we provide a detailed discussion of the profiling-derived parameters used in the performance models, as summarized in Table 2. Among these parameters, the memory bandwidths BW_{HBM} and BW_{L2}, the cube computation parameters L_{block} and II_{block}, and the format conversion coefficient α are architecture-specific to the Ascend NPU. The vector parameters L_{block}, II_{intra}, and II_{inter} further depend on the basic operator type, since different operators exhibit distinct execution latencies on the vector core.

To better illustrate the cost of profiling, Table 2 summarizes the profiling time of different parameters. The parameters of the memory model require the longest profiling time due to the large number of factors that must be considered. First, for each tile configuration, multiple measurements are performed to verify the stability of the observed bandwidth. Second, different tile sizes are tested to capture the bandwidth variation across different data transfer volumes. Finally, since format conversion is performed during the data transfer process, its overhead cannot be measured directly. To estimate this cost, we conduct comparative profiling experiments. Specifically, we measure the execution time of data transfers both with and without format conversion under different tile sizes and transfer counts. By comparing these measurements, the format conversion coefficient α can be derived. As a result, the total profiling time for the memory model is 124.5 minutes. In contrast,

Table 2. Profiling-Derived Parameters

Performance Model	Parameters	Profiling Time (min)	Parameter Category
Memory Model	$BW_{HBM}, BW_{L2}, \alpha$	124.5	Architecture-specific
Cube Computation Model	L_{block}, II_{block}	2.4	
Vector Computation Model	$II_{intra}, II_{inter}, L_{block}$	3.2	Operator-dependent

the cube computation model requires only 2.4 minutes, as it simply measures the latency of a single $16 \times 16 \times 16$ matrix multiplication instruction and the issue interval II_{block} between two such instructions. Similarly, the vector computation model requires only 3.2 minutes to profile L_{block}, II_{intra}, and II_{inter} for each basic operator.

5 Design Space Exploration

5.1 Problem Formulation

Leveraging the proposed performance models, we employ a DSE algorithm to automatically determine optimal parameter configurations under specified hardware constraints. These constraints, primarily determined by the memory capacities of the L1 cache and the L0A/B/C caches, serve as inputs to the DSE process. We formulate the resource-constrained performance optimization problem as

$$\begin{aligned} \underset{x}{minimize} \quad & Latency(x) \\ subject\,to \quad & r_i(x) \leq c_i(x), i = L1\,cache, L0A/B/C\,caches \end{aligned} \tag{16}$$

where $Latency(x)$ is the performance model function and denotes the end-to-end latency when applied with the parameter configuration x; $r_i(x)$ denotes the resource usage of component i, which must not exceed its memory capacity c_i; and i enumerates the NPU on-chip memory components, namely the L1 cache and the L0A/B/C caches.

As discussed in Figure 3(b), three major parameters in the design space influence the overall latency. The first is the tile size, which primarily affects the computation latency of the cube core. As shown in Equation (2), the cube core latency is expressed as $L_{allblocks} + L_{allstalls}$. While the tile size does not affect $L_{allblocks}$, it directly impacts $L_{allstalls}$ by changing the number of tiles and thus the number of data transfers from HBM. Smaller tiles increase the total tile count, leading to higher stall overhead and reduced computational efficiency. We also observed that the tile size has a negligible impact on the performance of the vector core. The remaining two parameters, buffer size and iteration direction, affect data transfer latency. The buffer size determines the amount of data movement, while the iteration direction influences the degree of data reuse. Both parameters affect the total data transfer volume and therefore the end-to-end latency.

5.2 Guided Search

The analysis of the design space parameters shows that operator performance is fundamentally determined by the volume of data movement. On the Ascend NPU architecture, most operators are memory bound, which means that data transfers across the memory hierarchy constitute the dominant bottleneck for end-to-end performance. Reducing data traffic is therefore essential for latency optimization. To capture this behavior, NPUMeter introduces a performance metric M_{data} to guide the exploration direction during the search process. M_{data} represents the total volume of data transferred from off-chip memory to on-chip memory during operator execution and is defined as

$$M_{data} = DataSize_{HBM} + DataSize_{L2Cache}, \tag{17}$$

where $DataSize_{HBM}$ and $DataSize_{L2Cache}$ denote the sizes of data that are stored in HBM and L2 cache, respectively.

The algorithm begins by initializing a set of candidate parameter configurations, including tile size, double buffer size, and iteration order. The tile size is set to 16 × 16 × 16, which corresponds to the minimum granularity supported by the cube core. The double buffer size is allocated accordingly, and the computational iteration order is initialized in a row-major manner. These initial settings form the starting point for performance model evaluation. Since the overall tile size is jointly determined by the three parameters and different combinations yield different latency results, we employ a heuristic search strategy. We begin by gradually increasing *baseK* to improve the utilization of L0A and L0B. Afterward, we reduce *baseK* and increase *baseM* and *baseN* to enhance the utilization of L0C while recording the configuration that achieves the lowest latency. This procedure repeats iteratively and increases the overall tile size at each step until hardware constraints are met or no further performance gains are detected.

In the second phase, we jointly optimize the buffer size and iteration order under the L1 cache capacity constraints. The double buffer size in L1 cache specifies how many tiles from the two input matrices can reside in L1 cache simultaneously, and its allocation must be carefully balanced due to the limited capacity of L1 cache. Moreover, the data reuse patterns of the two input matrices differ depending on the iteration order. For instance, when computing the first row of the output matrix, the first-row tiles of the left matrix are repeatedly reused, whereas the right matrix requires the entire matrix as input. To minimize the total data movement, we apply a greedy strategy to explore feasible double buffer sizes for each iteration order. Using the row-major iteration order as an example, we first increase the buffer size of the left matrix until it can hold a full row of tiles. After that, for the right matrix whose computation requires the entire matrix as input, we continue enlarging its buffer size in half-column increments so that as many of its tiles as possible can be stored. If the entire right matrix fits into L1 and additional memory space remains, we then allocate the remaining capacity to the left matrix. This process continues until either the L1 cache capacity is fully utilized or no further reduction in M_{data} can be achieved.

5.3 Generality Analysis

Although the above search procedure mainly targets parameters associated with matrix multiplication, the proposed search method is not limited to GEMM-like operators. Complex operators such as Attention, FFN, and GeGLU are typically decomposed into a sequence of matrix multiplications executed on the cube core and element-wise operations executed on the vector core. In practice, most performance optimization opportunities arise in the cube core computations, whose efficiency is highly sensitive to parameters such as tile size, double buffer size, and computational iteration order, forming a large design space. In contrast, the execution latency of the vector core is largely determined by the total computation volume and is independent of tiling configurations, as reflected in Equation (14). Therefore, the guided search primarily explores parameter configurations for matrix computations on the cube core. These parameters represent hardware execution characteristics, such as tiling granularity and on-chip memory allocation, rather than operator-specific semantics, allowing the same search strategy to be applied across different kinds of operators without introducing new operator-specific heuristics.

6 Evaluation

6.1 Experimental Setup

We evaluate NPUMeter on several operators that are widely used in the AI domain, including GEMM [21, 22], GeLU [23, 25], GeGLU [24, 25], **Feed-Forward Network (FFN)** [26, 27], and attention [20]. All experiments use FP16 data precision and are conducted on the Ascend 910B series NPU platform, specifically on a single AI core with a theoretical peak performance of 13.52 TFLOPS.

Table 3. Average Estimation Error of Performance Model

Operator	Input Shape (M, N, K)	Configuration Range			Average Estimation Error for Execution Time (%)				
		Cube Tile Size	DB Size in L1	Vec Tile Size	MTE2	MTE1	Cube	Vec	End-to-End
GEMM	(128,256,128)~ (4096,4096,4096)	(16,16,16)~ (128,256,64)	(16KB,32KB)~ (128KB,256KB)	-	3.04	0.41	0.23	-	3.31
GeLU	(128,256,128)~ (4096,4096,4096)	-	-	(128,128)~ (128,256)	3.63	-	-	0.65	3.95
GeGLU	(128,256,128)~ (4096,4096,4096)	(16,16,16)~ (128,256,64)	(16KB,32KB)~ (128KB,256KB)	(128,128)~ (128,256)	4.26	1.64	0.95	0.83	3.79
FFN	(128,256,128)~ (4096,4096,4096)	(16,16,16)~ (128,256,64)	(16KB,32KB)~ (128KB,256KB)	(128,128)~ (128,256)	4.46	1.27	0.62	0.65	3.17
Attention	(128,256,128)~ (4096,4096,4096)	(16,16,16)~ (128,256,64)	(16KB,32KB)~ (128KB,256KB)	(128,128)~ (128,256)	4.76	1.46	0.96	0.73	4.32

6.2 Performance Model Accuracy

To evaluate the estimation accuracy of our performance models in NPUMeter, we compare the execution time of operators estimated by NPUMeter with those collected from the onboard execution and compute the average estimation error for each operator across multiple input shapes and configurations. The detailed comparison is shown in Table 3. We evaluate five representative operators: cube operators involving matrix computations on the cube core (e.g., GEMM), vector operators involving vector computations on the vector core (e.g., GeLU), and hybrid operators that utilize both matrix and vector computations (e.g., GeGLU, FFN, and Attention). For each operator, we evaluate 180 distinct input shapes. These shapes are generated by exhaustively combining the three parameters M, N, and K. M and K range from 128 to 4096, while N ranges from 256 to 4096. Each parameter takes a value from the set of 256, 512, 1024, 2048, and 4096. For every input shape, we further evaluate multiple parameter configurations, including the tile sizes and double buffer size in L1 cache, as shown in Table 3. We also vary the iteration order (row-major or column-major) for all operators, which is omitted from Table 3.

We can see that NPUMeter accurately estimates the latency of each individual stage as well as the end-to-end execution time, achieving an average estimation error of less than 5% across a wide range of tensor shapes for all operators. Specifically, the computation model of NPUMeter for both the cube core and vector core achieves high estimation accuracy, with less than 1% estimation errors for all operators. This demonstrates that our pipelined computation models can precisely depict the execution process of the cube core and vector core. The error in end-to-end execution time primarily arises from inaccuracies in memory access time estimation, specifically due to the MTE2 stage, which handles data transfers from off-chip memory to the L1 cache. This is because, during MTE2, data may reside in either HBM or the L2 cache, depending on the inherent cache management strategy. However, accurately modeling this behavior is challenging due to the non-deterministic and opaque nature of this black-box cache management policy on target NPU. Despite this uncertainty, the MTE2 time can still be estimated with low errors, ranging from 2.48% to 4.86% across all operators.

To further investigate the effectiveness of our performance model, we select representative configurations for each operator, as plotted in Figures 11, 12, and 13. We present the actual onboard execution times, the estimated times returned by our performance models and the corresponding error rates of our models. We first evaluate the end-to-end performance of four operators with different configurations, as shown in Figure 11. The average estimation error remains within 5%, demonstrating strong overall prediction accuracy. To understand the source of end-to-end errors, Figure 12 and Figure 13 present detailed breakdown of the computation model and memory model, respectively. Figure 12 reports the computation time of Softmax, Tanh, GeLU, and GEMM on both the cube core and the vector core. NPUMeter achieves high estimation accuracy for both cores, with estimation errors below 1% for all operators. Figure 13 presents the memory access times of

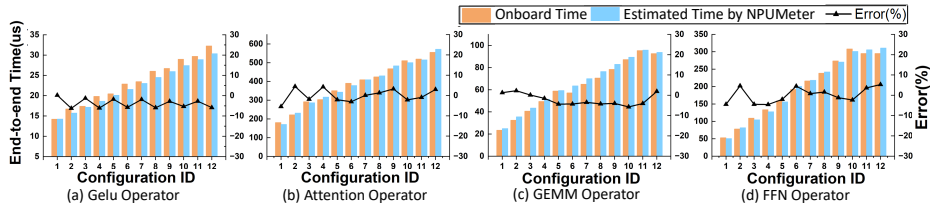


Fig. 11. Estimation accuracy of the end-to-end model.

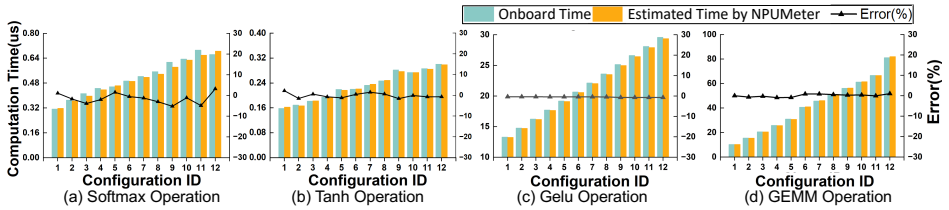


Fig. 12. Estimation accuracy of the computation model.

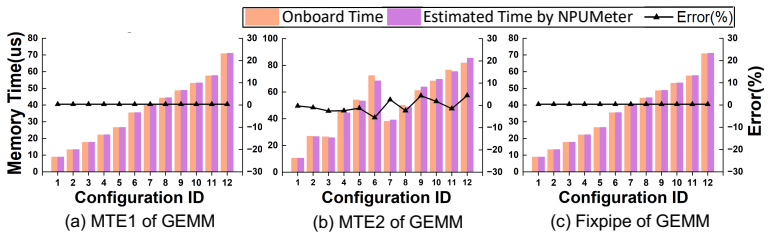


Fig. 13. Estimation accuracy of the memory model.

GEMM for MTE1, MTE2, and Fixpipe. The estimation error for MTE1 and Fixpipe remains relatively stable across configurations, while the error for MTE2 shows larger fluctuations, which stem from the black-box cache management policy of the L2 cache. This confirms that MTE2 is the dominant source of error in the overall end-to-end model.

We further provide more detailed results for these operators across several input shapes using the default configuration from the Ascend C compiler, as shown in Table 4. The table reports metrics including onboard execution time and NPUMeter-estimated time of cube core, vector core, and MTE2, the corresponding estimation errors, and the achieved cube FLOPS utilization. The average estimation errors remain below 2.5% for computation time and below 5% for both memory and end-to-end latency. Overall, NPUMeter delivers stable and consistently high estimation accuracy across a wide range of operators and parameter configurations, effectively predicting both end-to-end latency and the latencies of individual execution stages. Meanwhile, the cube-core executions achieve a high percentage of the theoretical peak FLOPS, indicating efficient utilization of the hardware compute resources.

6.3 DSE Results and Comparison

To validate the effectiveness of our DSE algorithm, Table 5 summarizes both the efficiency and the performance improvements achieved by the proposed approach across several representative operators and input shapes. The “Average Search Time” denotes the average runtime required for

Table 4. Detailed Results of Representative Input Shapes from Performance Models

Input Shape (M,N,K)	Operator	Cube(us)		Vec(us)		MTE2(us)		End-to-end(us)	
		On-board	NpuMeter	On-board	NpuMeter	On-board	NpuMeter	On-board	NpuMeter
(512,1024,128)	**GEMM**	10.24[96.97%]	10.24(0.00%)	-	-	10.49	10.46(-0.29%)	27.14	27.05(-0.33%)
	GeLU	-	-	31.23	30.76(-1.50%)	9.53	8.94(-6.19%)	34.25	35.42(3.42%)
	GeGLU	10.24[96.97%]	10.24(0.00%)	60.98	61.18(0.33%)	29.53	28.56(3.66%)	166.34	170.76(2.66%)
	FFN	20.86[95.18%]	20.48(-1.82%)	31.98	30.76(-3.81%)	49.35	50.15(1.62%)	230.35	225.64(-2.04%)
	Attention	21.43[92.68%]	20.48(-4.43%)	60.98	61.18(0.33%)	55.01	52.13(-5.24%)	263.59	258.46(-1.95%)
	Average Error	1.61%		1.19%		3.40%		2.08%	
(512,1024,256)	**GEMM**	20.53[96.73%]	20.48(-0.24%)	-	-	26.47	25.81(-2.49%)	44.96	44.61(-0.78%)
	GeLU	-	-	63.74	61.52(-3.48%)	20.34	18.32(-9.93%)	73.21	70.43(-3.80%)
	GeGLU	20.53[96.73%]	20.48(-0.24%)	62.31	61.52(-1.27%)	30.78	29.54(-4.03%)	195.13	198.51(1.73%)
	FFN	20.53[96.73%]	20.48(-0.24%)	64.15	61.52(-4.10%)	57.85	55.63(-3.84%)	218.35	220.31(0.90%)
	Attention	20.93[94.90%]	20.48(-2.15%)	61.38	61.18(-0.33%)	60.98	59.54(-2.36%)	291.66	281.81(-3.38%)
	Average Error	0.62%		2.30%		4.53%		2.12%	
(512,1024,384)	**GEMM**	30.98[96.15%]	30.72(-0.84%)	-	-	54.05	53.36(-1.28%)	62.77	61.05(-2.74%)
	GeLU	-	-	95.24	93.34(-1.99%)	29.64	29.54(-0.34%)	104.23	100.24(-3.83%)
	GeGLU	30.98[96.15%]	30.72(-0.84%)	60.98	61.18(0.33%)	69.45	69.47(0.03%)	231.25	210.35(-9.04%)
	FFN	30.98[96.15%]	30.72(-0.84%)	60.98	61.18(0.33%)	130.94	128.51(-1.86%)	261.86	256.13(-2.19%)
	Attention	31.28[95.24%]	30.72(-1.79%)	60.98	61.18(0.33%)	132.26	139.51(5.48%)	318.74	309.37(-2.94%)
	Average Error	1.03%		0.75%		1.80%		4.15%	
(512,1024,512)	**GEMM**	40.6[97.83%]	40.96(0.89%)	-	-	38.09	39.06(2.55%)	60.10	63.51(5.67%)
	GeLU	-	-	129.43	125.64(-2.93%)	41.25	40.64(-1.48%)	143.65	140.52(-2.18%)
	GeGLU	40.6[97.83%]	40.96(0.89%)	60.98	61.18(0.33%)	74.93	75.57(0.85%)	215.31	202.61(-5.90%)
	FFN	40.6[97.83%]	40.96(0.89%)	60.98	61.18(0.33%)	143.61	141.51(-1.46%)	251.63	250.31(-0.52%)
	Attention	42.97[92.43%]	40.96(-4.68%)	60.98	61.18(0.33%)	145.64	148.49(1.96%)	319.02	303.12(-4.99%)
	Average Error	1.65%		0.98%		1.66%		3.85%	
(512,1024,768)	**GEMM**	61.23[97.30%]	61.44(0.34%)	-	-	68.35	69.58(1.80%)	90.38	89.40(-1.08%)
	GeLU	-	-	160.25	157.26(-1.87%)	52.42	50.26(-4.12%)	180.32	173.67(-3.69%)
	GeGLU	61.23[97.30%]	61.44(0.34%)	60.98	61.18(0.33%)	131.01	134.09(2.35%)	267.54	256.89(-3.98%)
	FFN	61.23[97.30%]	61.44(0.34%)	60.98	61.18(0.33%)	259.39	261.48(0.81%)	294.65	300.54(2.00%)
	Attention	66.91[89.03%]	67.03(0.18%)	60.98	61.18(0.33%)	276.54	280.68(1.50%)	377.70	362.49(-4.03%)
	Average Error	0.31%		0.72%		2.12%		2.96%	

For the Cube Stage, Values in Brackets [] Indicate the Achieved Percentage of the Theoretical Peak FLOPS Measured on Hardware. For the NpuMeter Results, Values in Parentheses () Denote the Relative Prediction Error Compared with On-Board Measurements.

search methods to complete exploration across multiple input shapes of the same operator. For the on-board brute-force method, this time corresponds to generating, executing, and measuring all design points directly on the NPU. For NPUMeter, the reported time refers to the runtime of the proposed search algorithm, which terminates when either no further performance improvement is observed during exploration or when the explored configurations reach hardware resource limits (e.g., L1 cache capacity constraints). For search-time efficiency, the brute-force approach incurs substantial compilation, execution, and performance data collection overhead. Because it must repeatedly compile and execute every candidate configuration on hardware, its total search time is significantly prolonged. In contrast, the DSE method relies on analytical performance modeling and guided exploration, avoiding repeated on-board executions. Experimental results show that for operators with large design spaces, the proposed DSE algorithm reduces the overall search time from several hours (brute force) to only a few minutes. We further compare the operator performance using the DSE-generated configuration against the default configuration from the Ascend C compiler. Across a wide range of operators and input shapes, NPUMeter consistently achieves notable performance gains, delivering up to 1.46× speedup over the Ascend C compiler configuration. Moreover, even for regular input shapes (where all dimensions are powers of two) that are already well optimized by the Ascend C Compiler, the performance achieved with DSE-generated parameters remains competitive with the default configuration.

To better demonstrate the advantages of the proposed search method, we compare NPUMeter against state-of-the-art general-purpose automated tuning frameworks, including Halide [45] and Ansor [46]. Halide employs a beam search algorithm, while Ansor utilizes hierarchical sketch generation followed by an evolutionary search algorithm. In our experiments, we adopt their search strategies but evaluate candidate configurations using our own analytical performance model, ensuring a fair comparison focused on the search algorithm itself. The comparison results

Table 5. Comparison of DSE Results and Achieved Performance Across Various Operators and Input Shapes

Operator	Average Search Time (min)		Input Shape (M,N,K)	Design Space	Performance of Ascend Compiler Config. (us)	Performance of NPUMeter Config. (us) with Speedup
	On-board Brute Force	DSE of NPUMeter				
GEMM	136.5	14.5	(256,1792,1024)	2^{10}	141.67	97.07 (1.46×)
			(512,1792,1024)	2^{10}	206.74	177.84 (1.16×)
			(512,1024,256)	2^{11}	44.96	44.96 (1.00×)
GeGLU	170.7	15.3	(256,1792,1024)	2^{11}	215.79	162.25 (1.33×)
			(512,1792,1024)	2^{11}	503.25	435.26 (1.16×)
			(512,1024,256)	2^{12}	198.51	197.65 (1.00×)
FFN	213.3	18.7	(256,1792,1024)	2^{11}	350.63	280.51 (1.25×)
			(512,1792,1024)	2^{11}	540.67	465.31 (1.16×)
			(512,1024,256)	2^{12}	220.31	221.94 (0.99×)
Attention	273.1	20.2	(256,1792,1024)	2^{11}	389.09	317.81 (1.22×)
			(512,1792,1024)	2^{11}	615.29	579.27 (1.06×)
			(512,1024,256)	2^{12}	291.66	289.44 (1.01×)

Speedup is calculated through dividing the execution time of the default ascend compiler configuration by the execution time of the configuration returned by NPUMeter.

Table 6. Comparison Against General-Purpose Tuning Frameworks

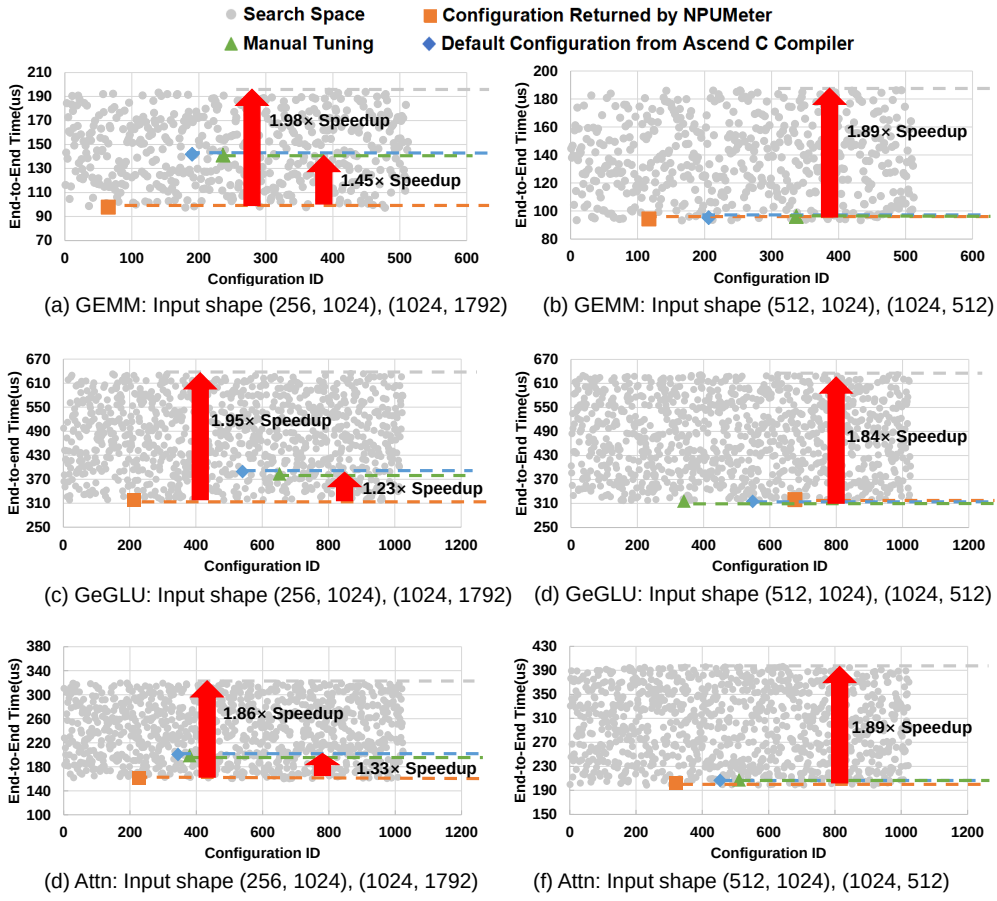
Operator	Average Search Time (min)			Perf. of Search Results (us)		
	Ansor [46]	Halide [45]	DSE of NPUMeter	Ansor [46] Config.	Halide [45] Config.	NPUMeter Config.
GEMM	18.5	27.4	14.5	100.35 (1.41×)	98.01 (1.45×)	97.07 (1.46×)
				177.41 (1.17×)	178.23 (1.16×)	177.84 (1.16×)
				46.26 (0.97×)	45.17 (1.00×)	44.96 (1.00×)
GeGLU	20.3	29.3	15.3	165.23 (1.31×)	163.89 (1.32×)	162.25 (1.33×)
				436.71 (1.15×)	434.97 (1.16×)	435.26 (1.16×)
				197.26 (1.01×)	198.03 (1.00×)	197.65 (1.00×)
FFN	23.7	35.9	18.7	281.09 (1.25×)	280.87 (1.25×)	280.51 (1.25×)
				467.93 (1.16×)	466.38 (1.16×)	465.31 (1.16×)
				220.98 (1.00×)	221.46 (0.99×)	221.94 (0.99×)
Attention	27.9	39.7	20.2	320.16 (1.22×)	318.92 (1.22×)	317.81 (1.22×)
				585.16 (1.05×)	580.16 (1.06×)	579.27 (1.06×)
				290.16 (1.01×)	289.96 (1.01×)	289.44 (1.01×)

are presented in Table 6, using the same operator settings and input shapes as in Table 5. The performance speedups are compared against the Ascend C compiler configuration in Table 5. The results demonstrate that NPUMeter consistently achieves comparable or even superior performance compared to these search methods, while requiring significantly less DSE time. This advantage comes from our guided search heuristic, which focuses on minimizing data movement and uses an effective stopping criterion to ensure exploration proceeds in the correct direction. By leveraging the white-box analytical model to evaluate candidate configurations, NPUMeter rapidly converges to high-quality configurations with far fewer evaluations, achieving much higher search efficiency on the NPU compared to general-purpose search strategies.

6.4 Optimality Analysis

To evaluate the quality of parameter configurations found by the DSE algorithm of NPUMeter, we perform a case study on three distinct operators, each evaluated with two representative input shapes, as shown in Figure 14. The search space is obtained by exhaustively testing each configuration onboard. We compare three baselines: (1) the worst-case latency; (2) the default parameter configuration from the Ascend C compiler; and (3) the optimized parameter configuration, which is tuned manually by experienced external engineers over several days (*manual tuning*).

For input matrices with regular shapes (where all dimensions are powers of two) in Figure 14(b)(d)(f), NPUMeter achieves performance comparable to both manual tuning and default



(a) GEMM: Input shape (256, 1024), (1024, 1792)
(b) GEMM: Input shape (512, 1024), (1024, 512)
(c) GeGLU: Input shape (256, 1024), (1024, 1792)
(d) GeGLU: Input shape (512, 1024), (1024, 512)
(d) Attn: Input shape (256, 1024), (1024, 1792)
(f) Attn: Input shape (512, 1024), (1024, 512)

Fig. 14. Optimality analysis.

configuration, achieving up to 1.89× speedup over the worst-case configuration. However, when any matrix dimension deviates from a power-of-two size, these baselines lead to significantly degraded performance. As demonstrated in Figure 14(a)(c)(e) for such irregular cases, NPUMeter delivers up to a 1.45× speedup over both manual tuning and the default configuration from Ascend C compiler, highlighting its robustness in optimality across diverse input shapes.

To further verify that our method does not get trapped in local optima, we conducted an experimental comparison with a genetic algorithm [47] in Table 7, which explores the design space by iteratively generating candidate configurations through mutation and crossover and selecting high-performing candidates for the next generation. The results show that NPUMeter achieves comparable or better configurations while achieving nearly 2× higher search efficiency, which further demonstrates that our method can efficiently converge to near-optimal solutions in the design space.

7 Portability to Other NPUs

Although NPUMeter is developed and validated on the Ascend Da Vinci architecture, we emphasize that the modeling methodology and several key insights are generalizable, while some implementation details are architecture-specific.

Generalizable insights. The core contribution of NPUMeter is a pipeline-based analytical modeling methodology that estimates end-to-end operator latency by analyzing the interactions,

Table 7. Comparison Against the Genetic Algorithm

Input Shape	Operator	Search Time (min)		Performance of Search Result (us)	
		Random Search	NPUMeter	Random Search	NPUMeter
(512,1792,1024)	GEMM	30.3	14.2	180.94	177.84
	GeGLU	35.1	15.2	45.79	44.96
(512,1024,256)	GEMM	30.5	14.7	438.31	435.26
	GeGLU	35.9	15.5	197.52	197.64

overlap, and stall behavior among different execution stages. This abstraction is broadly applicable to modern spatial accelerators, since many NPUs adopt a decoupled execution paradigm where data movement engines operate independently from large compute arrays. Moreover, the model captures the dataflow relationships among common operators such as matrix multiplication and element-wise operations, which are widely used in dataflow accelerators. Therefore, the overall modeling and analysis methodology can be reused when studying other NPU architectures.

Ascend-specific components. Some parts of NPUMeter are tied to the Ascend architecture and would require adaptation when porting to other NPUs. For example, the ND-to-Nz data layout conversion overhead is specific to Ascend NPUs and would not apply to architectures that natively support standard data layouts. In addition, architecture-dependent parameters such as memory bandwidth must be re-profiled for the target hardware.

Porting effort. Applying NPUMeter to a different NPU architecture requires analyzing the target accelerator's pipeline organization, memory hierarchy, and execution model, and mapping them to the stages in our analytical model, or slightly making adjustments based on our generalizable insights. After that, architecture-specific parameters need to be obtained through profiling on the target platform. For Ascend NPU variants (e.g., Ascend 310), this process is relatively straightforward since these chips follow the same Da Vinci architectural paradigm. For other NPUs, different instruction sets and compilation toolchains may require different profiling methods to extract the necessary parameters. Nevertheless, the profiling methodology described for Ascend NPUs in this work provides a reference for extracting the necessary parameters when adapting the model to new spatial architectures.

8 Related Work

With the rapid evolution of AI processors, research has increasingly focused on harnessing their computational capabilities across diverse applications and optimizing their hardware utilization. Existing studies can be broadly categorized into two directions: performance modeling, which aims at characterizing and predicting system behavior, and design space exploration, which seeks to identify optimal mapping and scheduling strategies.

8.1 Performance Model

Some research employs techniques tailored to specific hardware architectures. Similar to earlier studies on regular GPUs [34], Jia et al. use microbenchmarking to analyze the architecture of Tensor Cores [35]. Other studies focus on performance modeling for AI processors. For example, Raihan et al. [36] propose a Tensor Core microarchitecture model and deploy it on the GPGPU-Sim simulator. Chen et al. [38] construct a statistical cost model to estimate the cost of each low-level tensor program. Similarly, Halide [45] utilizes a machine learning-based cost model to predict the execution time of candidate schedules. These approaches typically treat the underlying hardware as a black box and are difficult to adapt to NPUs.

In addition, several works attempt to model FPGA architectures. COMBA [39] proposes an accurate analytical model for FPGA designs. Chen et al. develop a performance model for large language model inference on FPGAs [40]. There are also studies that concentrate on NPUs. Kaufman

et al. [37] build a performance model based on a graph neural network for tensor programs and achieve 96.3% and 95.5% accuracy with respect to real measurements on Google TPUs. WMDRS [7] targets Ascend NPUs with a deep learning-based model that predicts task execution time using real-time metrics such as AI Core utilization, frequency, and memory bandwidth. Our work focuses on the DaVinci Processor hardware details and proposes an analytical performance model with high accuracy.

8.2 DSE Algorithm

To better exploit the hardware performance of AI accelerators, some work focuses on DSE algorithms to explore optimal performance parameter configurations. Zhao et al. [44] perform directive optimization for high-level synthesis. POM [41] provides a heuristic-based DSE method to iteratively optimize bottleneck loops. StreamHLS [43] utilizes a nonlinear programming method to explore parallelism between tasks. These works are tailored for FPGA backends. Several other works focus on mapping problems for DNN accelerators. ZigZag [28] systematically enlarged the mapping space by introducing uneven mappings. To navigate this vast space efficiently, subsequent research focused on high-speed schedulers. Ansor [46] adopts a sketch-based search strategy to automatically generate candidate scheduling structures and explore their configurations. LOMA [29] offered a fast, deterministic approach with optimality guarantees, while SALSA's [30] dual-engine strategy dynamically balanced exhaustive and heuristic search for robust performance. CoSA [31] formulated the problem as a constrained optimization, using Mixed-Integer Programming (MIP) to solve for multiple mapping variables simultaneously. Most recently, Ceiba [32] has leveraged Deep Reinforcement Learning (DRL) with a tailored action space to generate high-performance schedules with superior scalability and adaptability.

8.3 Frameworks for Kernel Optimization

Recent work has explored compiler frameworks for optimizing accelerator kernels [49, 50, 53, 54]. Roller [54] proposes a hardware-aware tensor compilation framework that prunes the optimization space using explicit architectural constraints such as cache capacity and memory transaction alignment. AKG [53] targets NPUs and leverages polyhedral transformations to automatically generate optimized kernels, including memory promotion and tensorization for specialized accelerator instructions. Triton [49] and Helion [50] introduce Python-based domain-specific languages (DSLs) that simplify tile-level programming and memory management, primarily targeting GPU architectures. In contrast to these compiler frameworks that focus on programming abstractions and kernel generation, NPUMeter is designed as an analytical performance modeling and auto-tuning framework targeting Ascend NPUs.

Furthermore, recent studies have explored LLM-based kernel optimization [51, 52]. AccelOpt [51] and KernelEvolve [52] employ large language model (LLM) agents and retrieval-augmented generation to iteratively synthesize and refine hardware kernels. While these approaches demonstrate promising capabilities for automated kernel generation, they typically rely on extensive exploration and hardware feedback. In contrast, our analytical modeling approach provides interpretable performance insights and can guide optimization more efficiently. We argue that LLMs can be utilized to synthesize the initial functional kernel structures, while NPUMeter serves as a deterministic, white-box evaluator to instantaneously fine-tune the hardware configurations, completely bypassing the prohibitive overhead of on-board executions.

9 Conclusion

This article presents NPUMeter, an automatic operator optimization framework for NPU based on accurate and comprehensive analytical performance models. NPUMeter consists of two key

components: analytical performance model and a DSE search algorithm. The performance model can accurately model the computation, memory access, and end-to-end performance of NPU operators. The DSE algorithm rapidly explores the design space guided by performance metric. Experiments demonstrate that the performance model can achieve a high degree of accuracy, with an average estimation error of within 5%. Additionally, the DSE algorithm can not only achieve up to a 1.46× performance speedup compared to the configuration generated by the Ascend C compiler but also search for near-optimal configurations in just a few minutes.

Acknowledgments

We sincerely thank all the reviewers for their valuable comments.

References

[1] David Goodwin and Soyoung Jeong. 2019. Maximizing utilization for data center inference with TensorRT inference server. In *Proceedings of NVIDIA GPU Technology Conference (GTC)*, Session S9438. Retrieved May 2026 from https://developer.nvidia.com/gtc/2019/video/s9438

[2] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit[C]. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 1–12.

[3] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2014. Dadiannao: A machine-learning supercomputer[C]. In *Proceedings of the 2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 609–622.

[4] Y. Choi and M. Rhu. 2020. Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units[C]. In *Proceedings of the 2020 IEEE International Symposium on High Performance Computer Architecture*. IEEE, 220–233.

[5] E. Baek, D. Kwon, and J. Kim. 2020. A multi-neural network acceleration architecture[C]. In *Proceedings of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture*. IEEE, 940–953.

[6] Chengsi Gao, Ying Wang, Cheng Liu, Mengdi Wang, Weiwei Chen, Yinhe Han, and Lei Zhang. 2023. Layer-puzzle: Allocating and scheduling multi-task on multi-core NPUs by using layer heterogeneity[C]. In *Proceedings of the 2023 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1–6.

[7] Chong Liu, Yuan Yao, Yi Dang, Gang Yang, Wei Jia, Xinyu Tian, and Xingshe Zhou. 2023. WMDRS: Workload-aware performance model based multi-task dynamic-quota real-time scheduling for neural processing units[C]. In *Proceedings of the 2022 IEEE 28th International Conference on Parallel and Distributed Systems*. IEEE, 435–442.

[8] Zhe Jia, Blake Tillman, Marco Maggioni, and Daniele Paolo Scarpazza. 2019. Dissecting the Graphcore IPU Architecture via Microbenchmarking. *CoRR* abs/1912.03413 (2019). Retrieved from https://arxiv.org/abs/1912.03413

[9] Kamil Rocki, Dirk Van Essendelft, Ilya Sharapov, Robert Schreiber, Michael Morrison, Vladimir Kibardin, Andrey Portnoy, Jean Francois Dietiker, Madhava Syamlal, and Michael James. 2020. Fast stencil-code computation on a wafer-scale processor. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE Press, Article 58, 1–14.

[10] H. Liao, J. Tu, J. Xia, and X. Zhou. 2019. DaVinci: A scalable architecture for neural network computing. *In Proceedings of the 2019 IEEE Hot Chips 31 Symposium*. Cupertino, CA, USA, 1–44. DOI: 10.1109/HOTCHIPS.2019.8875654

[11] Eric Chung, Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Adrian Caulfield, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Maleen Abeydeera, Logan Adams, Hari Angepat, Christian Boehn, Derek Chiou, Oren Firestein, Alessandro Forin, Kang Su Gatlin, Mahdi Ghandi, Stephen Heil, Kyle Holohan, Ahmad El Husseini, Tamas Juhasz, Kara Kagi, Ratna K. Kovvuri, Sitaram Lanka, Friedel van Megen, Dima Mukhortov, Prerak Patel, Brandon Perez, Amanda Rapsang, Steven Reinhardt, Bita Rouhani, Adam Sapek, Raja Seera, Sangeetha Shekar, Balaji Sridharan, Gabriel

Weisz, Lisa Woods, Phillip Yi Xiao, Dan Zhang, Ritchie Zhao, and Doug Burger. 2018. Serving DNNs in real time at datacenter scale with project brainwave. *IEEE Micro* 38, 2 (2018), 8–20. DOI: https://doi.org/10.1109/MM.2018.022071131

[12] Quan Chen, Hailong Yang, Minyi Guo, Ram Srivatsa Kannan, Jason Mars, and Lingjia Tang. 2017. Prophet: Precise QoS prediction on non-preemptive accelerators to improve utilization in warehouse-scale computers. *SIGPLAN Not.* 52, 4 (2017), 17–32. DOI: https://doi.org/10.1145/3093336.3037700

[13] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: QoS awareness and increased utilization for non-preemptive accelerators in warehouse scale computers. *SIGPLAN Not.* 51, 4 (2016), 681–696. DOI: https://doi.org/10.1145/2954679.2872368

[14] Jonathan Lew, Deval A. Shah, Suchita Pati, Shaylin Cattell, Mengchi Zhang, Amruth Sandhupatla, Christopher Ng, Negar Goli, Matthew D. Sinclair, Timothy G. Rogers, and Tor M. Aamodt. 2019. Analyzing machine learning workloads using a detailed GPU simulator. *In Proceedings of the 2019 IEEE International Symposium on Performance Analysis of Systems and Software.* Madison, WI, USA, 151–152. DOI: https://doi.org/10.1109/ISPASS.2019.00028

[15] Yuan Yao, Shuangyang Liu, Sikai Wu, Jinyu Wang, Jinting Ni, Gang Yang, and Yu Zhang. 2022. WAMP^{2}2S: Workload-aware GPU performance model based pseudo-preemptive real-time scheduling for the airborne embedded system. In *IEEE Transactions on Parallel and Distributed Systems* 33, 11 (2022), 2767–2780. DOI: https://doi.org/10.1109/TPDS.2021.3134269

[16] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to DNN accelerator evaluation. In *Proceedings of the 2019 IEEE International Symposium on Performance Analysis of Systems and Software.* Madison, WI, USA, 304–315. DOI: https://doi.org/10.1109/ISPASS.2019.00042

[17] S.-C. Kao and T. Krishna. 2020. GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm. In *Proceedings of the 2020 IEEE/ACM International Conference On Computer Aided Design.* San Diego, CA, USA, 1–9.

[18] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). *CoRR*, abs/1606.08415. Retrieved from https://arxiv.org/abs/1606.08415

[19] Noam Shazeer. 2020. GLU variants improve transformer. *CoRR*, abs/2002.05202. Retrieved from https://arxiv.org/abs/2002.05202

[20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*, Kaiser and Illia Polosukhin (Eds.). 30 (2017), 6000–6010.

[21] P. Springer, P. Bientinesi.2018. Design of a high-performance GEMM-like tensor–tensor multiplication[J]. *ACM Transactions on Mathematical Software* 44, 3 (2018), 1–29.

[22] B. Kågström, P. Ling, C. Van Loan. 1998. GEMM-based level 3 BLAS: high-performance model implementations and performance evaluation benchmark[J]. *ACM Transactions on Mathematical Software* 24, 3 (1998), 268–302.

[23] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). *CoRR*, abs/1606.08415. Retrieved from https://arxiv.org/abs/1606.08415

[24] Noam Shazeer. 2020. GLU Variants Improve Transformer. *CoRR* abs/2002.05202 (2020). https://arxiv.org/abs/2002.05202

[25] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy, Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikuła, Mateo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L. Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimenko, Tom Hennigan, Vlad Feinberg, Wojciech Stokowiec, Yu-hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. 2024. Gemma: Open Models Based on Gemini Research and Technology. *CoRR* abs/2403.08295 (2024). https://arxiv.org/abs/2403.08295

[26] D. Svozil, V. Kvasnicka,J. Pospichal. 1997. Introduction to multi-layer feed-forward neural networks[J]. *Chemometrics and Intelligent Laboratory Systems* 39, 1 (1997), 43–62.

[27] G. Bebis, M. Georgiopoulos. 2021. Feed-forward neural networks[J]. *Ieee Potentials* 13, 1 (1994), 27–31.

[28] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. 2021. Verhelst. ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators. In *IEEE Transactions on Computers* 70, 8 (2021), 1160–1174. DOI: https://doi.org/10.1109/TC.2021.3059962

[29] A. Symons, L. Mei and M. Verhelst. 2021. LOMA: Fast auto-scheduling on DNN accelerators through loop-order-based memory allocation. In *Proceedings of the 2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems*. Washington DC, DC, USA, 1–4. DOI: https://doi.org/10.1109/AICAS51828.2021.9458493

[30] V. J. B. Jung, A. Symons, L. Mei, M. Verhelst, and L. Benini. 2023. Benini. SALSA: Simulated annealing based loop-ordering scheduler for DNN accelerators. In *Proceedings of the 2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems*. Hangzhou, China, 1–5. DOI: https://doi.org/10.1109/AICAS57966.2023.10168625

[31] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. CoSA: Scheduling by constrained optimization for spatial accelerators. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture*. Valencia, Spain, 554–566. DOI: https://doi.org/10.1109/ISCA52012.2021.00050

[32] Fuyu Wang, Minghua Shen, Yutong Lu, and Nong Xiao. 2025. Ceiba: An efficient and scalable DNN scheduler for spatial accelerators. *ACM Transactions on Architecture and Code Optimization* 22, 2 (2025), 23 pages. DOI: https://doi.org/10.1145/3715123

[33] Yifeng Tang and Cho-li Wang. 2023. Performance modeling on DaVinci AI core. *Journal of Parallel and Distributed Computing* 175 (2023), 134–149. DOI: https://doi.org/10.1016/j.jpdc.2023.01.008

[34] Xiuxia Zhang, Guangming Tan, Shuangbai Xue, Jiajia Li, Keren Zhou, and Mingyu Chen. 2017. Understanding the GPU microarchitecture to achieve bare-metal performance tuning. *SIGPLAN Not.* 52, 8 (2017), 31–43. DOI: https://doi.org/10.1145/3155284.3018755

[35] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. 2018. Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking. *CoRR* abs/1804.06826 (2018). Retrieved from https://arxiv.org/abs/1804.06826

[36] M. A. Raihan, N. Goli, and T. M. Aamodt. 2019. Modeling deep learning accelerator enabled GPUs. *In Proceedings of the 2019 IEEE International Symposium on Performance Analysis of Systems and Software*. Madison, WI, USA, 79–92. DOI: https://doi.org/10.1109/ISPASS.2019.00016

[37] Samuel J. Kaufman, Phitchaya Mangpo Phothilimthana, Yichen Zhou, and Mike Burrows. 2020. A Learned Performance Model for the Tensor Processing Unit. *CoRR* abs/2008.01040 (2020). Retrieved from https://arxiv.org/abs/2008.01040

[38] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, 3393–3404.

[39] Jieru Zhao, Liang Feng, Sharad Sinha, Wei Zhang, Yun Liang, and Bingsheng He. 2017. COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications. In *Proceedings of the 2017 IEEE/ACM International Conference on Computer-Aided Design*. IEEE Press, 430–437. DOI: https://doi.org/10.1109/ICCAD.2017.8203809

[40] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the potential of FPGA-based spatial acceleration for large language model inference. *ACM Transactions on Reconfigurable Technology and Systems* 18, 1 (2025), 29 pages. DOI: https://doi.org/10.1145/3656177

[41] Weichuang Zhang, Jieru Zhao, Guan Shen, Quan Chen, Chen Chen, and Minyi Guo. 2024. An optimizing framework on MLIR for efficient FPGA-based accelerator generation. In *Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 75–90. DOI: https://doi.org/10.1109/HPCA57654.2024.00017

[42] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2022. ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation. In *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture*.

[43] Suhail Basalama and Jason Cong. Stream-HLS: Towards Automatic Dataflow Acceleration. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 103–114. DOI: https://doi.org/10.1145/3706628.3708878

[44] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He. 2020. Performance modeling and directives optimization for high-level synthesis on FPGA. In *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 7 (2020), 1428–1441. DOI: https://doi.org/10.1109/TCAD.2019.2912916

[45] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, and Jonathan Ragan-Kelley. 2019. Learning to optimize halide with tree search and random programs. *ACM Transactions on Graphics* 38, 4 (2019), 12 pages. DOI: https://doi.org/10.1145/3306346.3322967

[46] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. USENIX Association, USA, Article 49, 863–879.

[47] David E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization and Machine Learning (1st. ed.).* Addison-Wesley Longman Publishing Co. USA.

[48] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. *In Proceedings of the 32nd International Conference on Neural Information Processing Systems.* Curran Associates Inc., Red Hook, NY, USA, 3393–3404.

[49] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages.* Association for Computing Machinery, New York, NY, USA, 10–19. DOI: https://doi.org/10.1145/3315508.3329973

[50] J. Ansel, O. Ulgen, W. Feng, J. Choi, and E. Ellison. 2025. Helion: A high-level DSL for performant and portable ML kernels. *PyTorch Official Blog*, Meta, Oct. 2025. [Online]. Available: Retrieved from https://pytorch.org/blog/helion/

[51] Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. 2026. AccelOpt: A self-improving LLM agentic system for AI accelerator kernel optimization. In *Proceedings of Machine Learning and Systems (MLSys).*

[52] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, Zewei Jiang, Dianshi Li, Uladzimir Pashkevich, Varna Puvvada, Feng Shi, Matt Steiner, Ruichao Xiao, Nathan Yan, Xiayu Yu, Zhou Fang, Roman Levenstein, Kunming Ho, Haishan Zhu, Alec Hammond, Richard Li, Ajit Mathews, Kaustubh Gondkar, Abdul Zainul-Abedin, Ketan Singh, Hongtao Yu, Wenyuan Chi, Barney Huang, Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook, Carole-Jean Wu, and Gaoxiang Liu. 2026. KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta. *CoRR* abs/2512.23236 (2026). Retrieved from https://arxiv.org/abs/2512.23236

[53] Jie Zhao, Bojie Li, Wang Nie, Zhen Geng, Renwei Zhang, Xiong Gao, Bin Cheng, Chen Wu, Yun Cheng, Zheng Li, Peng Di, Kun Zhang, and Xuefeng Jin. 2021. AKG: Automatic kernel generation for neural processing units using polyhedral transformations. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI).* 1233–1248. DOI: https://doi.org/10.1145/3453483.3454106

[54] Haichen Zhu, Ruihang Yao, Mingzhe Zhai, et al. 2022. Roller: Fast and efficient tensor compilation for deep learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI).* 233–248.

Received 6 December 2025; revised 10 March 2026; accepted 27 May 2026